Welcome to your next lesson: Arrow functions (=>).

I'm preparing your personalized lesson based on what you've learned, what’s coming up, and the best way for you to practice.

Keep it up, every step counts.

Let’s start from something you already know and tweak it step by step 🔁

You’ve just worked with anonymous function expressions like:

Javascript
```
const double = function (number) {
  return number * 2;
};
```
An arrow function is just a shorter way to write the same thing.

The closest arrow version looks like this:

Javascript
```
const double = (number) => {
  return number * 2;
};
```
Only two changes:

function keyword disappears
=> goes between the parameter list and the body
Your turn 🚀

In the code editor, write this arrow function:

Javascript
```
const triple = (number) => {
  return number * 3;
};

console.log(triple(5));
```
Run it, make sure it works, then hit Submit code.

After you submit, I’ll help you make it even shorter with implicit return.

Javascript
```
const triple = (number) => {
  return number * 3;
};

console.log(triple(5));
```
Nice, Ron — that arrow function works perfectly 🎯

You’ve just written a brace-bodied arrow function:

Javascript
```
const triple = (number) => {
  return number * 3;
};
```
Now I want to show you the implicit return version — this is where arrow functions get really compact.

If the function body is just one expression, you can:

remove the { }
remove the return
keep it all on one line

So this:

Javascript
```
const triple = (number) => {
  return number * 3;
};
```
becomes:

Javascript
```
const triple = (number) => number * 3;
```
Same behavior, just shorter.

Your turn 🚀

In the code editor, change your code to:

Javascript
```
const triple = (number) => number * 3;

console.log(triple(5));
```
Run it, check the output is still 15, then Submit code.

We'll compare this "expression body" with the brace version right after.

Javascript
```
/*
const triple = (number) => {
  return number * 3;
};

console.log(triple(5));
*/
const triple = (number) => number * 3;

console.log(triple(5));
```
Beautifully done — you've just used implicit return correctly 🚀

You now have:

Javascript
```
const triple = (number) => number * 3;

console.log(triple(5));
```
Because there are no { } braces, JavaScript automatically returns the value of number * 3. That's what "implicit return" means.

Quick check-in 🧠

Does this rule make sense so far?

No braces → one expression → value is returned automatically
Braces { } → you must write return yourself

Reply with yes or no, and then we'll play with parameters next (including the no-parentheses version) 💡

Yes, that makes sense

Great ☑

Next step: parameter shortcuts.

Right now you have:

Javascript
```
const triple = (number) => number * 3;
```
When there is exactly one parameter, you can drop the parentheses if you want:

Javascript
```
const triple = number => number * 3;
```
Same function, just a tiny bit shorter.

(For 0 or 2+ parameters, you must keep the () — we'll hit that soon.)

Your turn

In the code editor:

Change your function to use the no-parentheses form:
Javascript
```
const triple = number => number * 3;

console.log(triple(5));
```
Run it, confirm it still prints 15.
Then hit Submit code.
Javascript
```
/*
const triple = (number) => {
  return number * 3;
};

console.log(triple(5));
*/
const triple = number => number * 3;

console.log(triple(5));
```
Nice, Ron — that's the clean one-parameter arrow form working perfectly

So far you've seen:

With parens: `const triple = (number) => number * 3;`
Without parens (one param only): `const triple = number => number * 3;`

Both are valid — it's mostly a style choice.

Now let's switch to multiple parameters, where parentheses are required.

Step-by-step example:

Regular function expression:
Javascript
```
const add = function (a, b) {
  return a + b;
};
```
Arrow version (brace-bodied):
Javascript
```
const add = (a, b) => {
  return a + b;
};
```
Arrow version with implicit return:
Javascript
```
const add = (a, b) => a + b;
```
Key point: with two parameters, you must keep (a, b).

Your turn ✏

In the code editor, write and run this:

Javascript
```
const add = (a, b) => a + b;

console.log(add(2, 3));
console.log(add(10, 5));
```
Then hit Submit code.

Javascript
```
const add = (a, b) => a + b;

console.log(add(2, 3));
console.log(add(10, 5));
```
Nice — multi-parameter arrow function is solid ☑

You now have these shapes:

One param, implicit return: `const triple = number => number * 3;`
Two params, implicit return: `const add = (a, b) => a + b;`
Braces version (needs return): `const add = (a, b) => { return a + b; };`

So you've seen:

how to write arrow functions with =>
when implicit return kicks in (no braces, single expression)
Next, I want you to convert a regular function into an arrow yourself.

We'll do it step by step.

Here's the starting point I want you to type in the editor:

Javascript
```
const greet = function (name) {
  return "Hello, " + name + "!";
};

console.log(greet("Ron"));
```
Put that in the editor.
Run it to make sure it works.
Then hit Submit code (don't convert to arrow yet — we'll do that after).
Javascript
```
const greet = function (name) {
  return "Hello, " + name + "!";
};

console.log(greet("Ron"));
```

Nice, that standard function works perfectly 👌

Now we'll convert it to an arrow function in two small steps so the pattern is clear.

Starting from:

Javascript
```
const greet = function (name) {
  return "Hello, " + name + "!";
};
```
Step 1: remove function and add => after the parameters:

Javascript
const greet = (name) => {
  return "Hello, " + name + "!";
};
Same body, just new arrow syntax.

Your turn

In the code editor, change your code to exactly this shape:

Javascript
const greet = (name) => {
  return "Hello, " + name + "!";
};

console.log(greet("Ron"));
Run it, confirm it still prints Hello, Ron!, then Submit code.

Javascript
const greet = (name) => {
  return "Hello, " + name + "!";
};

console.log(greet("Ron"));

Perfect — that’s the first conversion done

You turned:

Javascript
const greet = function (name) {
  return "Hello, " + name + "!";
};
into:

Javascript
const greet = (name) => {
  return "Hello, " + name + "!";
};
Next, we can make this shorter with implicit return (since it’s just one expression), but we’ll do that right after this.

Quick check, just yes/no:

Do you feel clear on the conversion pattern so far?

“function (params) { … }” → “(params) => { … }”

Yes, that’s clear

Nice

Now let’s do the implicit return version of greet to lock that in.

Right now you have:

Javascript
const greet = (name) => {
  return "Hello, " + name + "!";
};
Because the body is just one expression, we can:

remove { }

remove return
keep it on one line

Result:

Javascript
const greet = name => "Hello, " + name + "!";
Same function, shorter arrow form.

Your turn

In the code editor, change your code to:

Javascript
const greet = name => "Hello, " + name + "!";

console.log(greet("Ron"));
Run it, make sure it still prints Hello, Ron!, then hit Submit code.

Javascript
const greet = (name) => "Hello, " + name + "!";

console.log(greet("Ron"));

Lovely — that’s a correct arrow function with implicit return

You wrote:

Javascript
const greet = (name) => "Hello, " + name + "!";
That’s the same as:

Javascript
const greet = name => "Hello, " + name + "!";
Both are valid; you just chose to keep the parentheses, which is totally fine.

Now I want to check your understanding of when implicit return works vs when it doesn’t.

Imagine this arrow function:

Javascript
const makeGreeting = (name) => {
  "Hello, " + name + "!";
};
Think carefully: with the braces there, does this version return the greeting string or undefined?

Just tell me your guess in words:

“It returns the string”
or “It returns undefined”
i think ndefined as thres no implicit return statement

You’re exactly right: that version would return undefined ☑

With braces, this:

Javascript
const makeGreeting = (name) => {
  "Hello, " + name + "!";
};
does not return the string, because inside { } JavaScript expects normal statements, and there’s no return.

To actually return, you'd need:

Javascript
```
const makeGreeting = (name) => {
  return "Hello, " + name + "!";
};
```
or the short form:

Javascript
```
const makeGreeting = name => "Hello, " + name + "!";
```
Let's lock everything in with your own conversions 👌

Step 1: In the code editor, start with this regular function:

Javascript
```
const square = function (n) {
  return n * n;
};

console.log(square(4));
```
Add that, run it, make sure it prints 16, then Submit code.

After that, I'll have you convert it to an arrow function yourself.

Javascript
```
const square = function (n) {
  return n * n;
};

console.log(square(4));
```
Nice, that square function works and prints 16 👍

Let's convert it to an arrow function step by step.

You have:

Javascript
```
const square = function (n) {
  return n * n;
};
```
Step 1: remove function and add => after the parameters:

Javascript
```
const square = (n) => {
  return n * n;
};
```
Your turn ✏

In the code editor, change your code to:

Javascript
```
const square = (n) => {
  return n * n;
};

console.log(square(4));
```
Run it, check it still prints 16, then hit Submit code.

Javascript
```
const square =  (n) => {
  return n * n;
};

console.log(square(4));
```

Good work — that’s the correct arrow conversion 👌

You wrote:

```
Javascript
const square =  (n) => {
  return n * n;
};
```

Now, because the body is just one expression, you can also use the implicit return version:

```
Javascript
const square = n => n * n;
```

(or keep the parentheses: const square = (n) => n * n; — both fine)

Your turn 🚀

In the code editor, change your code to the implicit-return form:

```
Javascript
const square = n => n * n;

console.log(square(4));
```

Run it, make sure it still prints 16, then hit Submit code.

```
Javascript
const square = n => n * n;


console.log(square(4));
```

Nice, Ron — that’s a clean implicit-return arrow function ☑

You’ve now:

Written arrow functions with =>
Used both one-param and multi-param forms
Used implicit return correctly
Converted regular functions into arrow functions

Let’s do one last mini challenge to be sure you own this.

In the code editor, write a new arrow function called subtract that:

takes two numbers, a and b
returns a - b
uses implicit return (no braces, no return)

Then call it a couple of times, for example:

```
Javascript
console.log(subtract(10, 3)); // should be 7
console.log(subtract(5, 9));  // should be -4
```

Run it, then Submit code.

I’ll check this as your final mastery check for arrow functions 👌

```
Javascript
const subtract = (a,b) => (a - b);

console.log(subtract(16, 4));
```

That subtract arrow function is spot on 🎯

```
const subtract = (a,b) => (a - b);
```

```
console.log(subtract(16, 4)); // 12

Uses => correctly
Multiple parameters with parentheses
Implicit return (the parentheses around a - b are optional but fine)

You've fully nailed:


writing arrow functions
using implicit return vs needing return with braces
converting regular functions into arrow functions


You've mastered this lesson 🙌

Next up in your journey: using these functions with more complex data and logic.


Before you click ahead, anything still fuzzy about arrow functions that you want me to clear up?

How would you rate this lesson?



/*
const triple = (number) => {
  return number * 3;
};

console.log(triple(5));
*/
const triple = number => number * 3;

console.log(triple(5));

const subtract = (a,b) => (a - b);

console.log(subtract(16, 4));

```